P4 Proposal — FirstPaycheck: a new-job money guide

Group: Group 2

Members: Ethan Luu, Documents and retrieval · Shawn Prather, Financial engine and agent tools · Kaelem Deng, User-facing app · Callen Nash, Evaluation, cost, and safety

1. Our user and their problem

Our user is the incoming new-hire cohort at Tyler Technologies, a local employer that starts a group of early-career hires this winter. Our contact is Nathan Woo, AI Developer, who speaks for the cohort and connects us with 6 new hires for interviews and testing.

The problem: a new hire receives an offer letter with a yearly salary, a long benefits guide, and a short enrollment window. Choices made in that window, such as the health plan, usually cannot change until the next open enrollment. Most first-time employees do not know:

- what their actual take-home pay per check will be after taxes and deductions
- which health plan, HSA or FSA, and insurance options fit their situation
- how much to contribute to the 401(k), whether to add a Roth IRA, and which comes first
- whether the company stock purchase plan is worth joining
- how to split each paycheck between bills, savings, debt, and investing
- whether to switch plans each November, when open enrollment lets them change

Today they guess, ask friends, or leave the default. Leaving money on the table, like an unclaimed employer match, is the common result. We spoke with our contact, Nathan, on the 27th of October; what we learned is reflected in the examples below. Nathan is now connecting us with new hires in the cohort, and we will talk with at least one before we submit.

2. Five real examples

Each example is a question a new hire asks, the document it depends on, and what a correct answer must contain. These also become the first cases in our evaluation set. All five are in full in Appendix A.

Hard case: Example 4. The obvious answer, that a 15% discount is free money so always buy, is wrong when the plan has a holding period or the new hire has no emergency fund yet.

3. What we will build

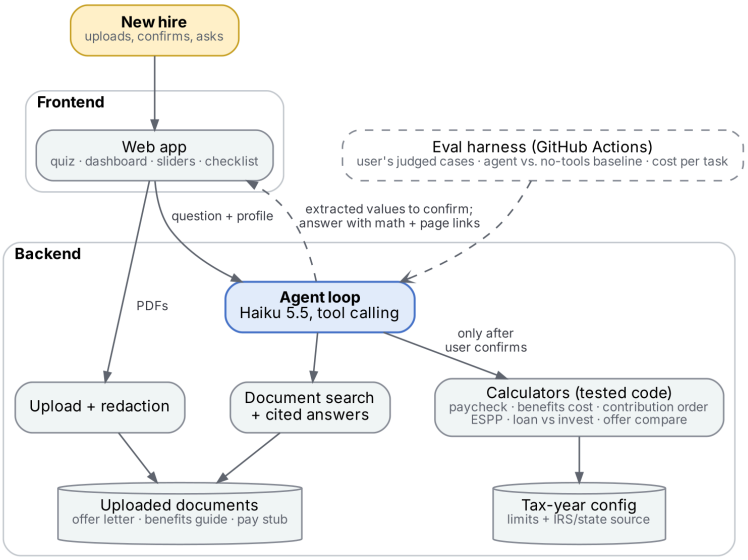
We will build a web app. Four core features ship first: a document upload and explainer that answers with links to the source page; a take-home pay calculator with every deduction itemized; a benefits enrollment advisor comparing yearly plan costs; and the 401(k) and Roth order of operations in dollars per check. Seven more build on them: a paycheck split plan, what-if sliders, an enrollment deadline checklist, a beginner glossary and chat, company stock (ESPP/RSU) guidance, student loans versus investing, and comparing two job offers. Section 5 says which we cut first.

What our user does and sees: Example 1

A new hire opens FirstPaycheck and uploads their offer letter. The agent reads the salary and pay frequency from the letter and shows them back with a link to the page they came from. The user confirms them, adds filing status and Colorado residence in the quiz, and asks: "What actually hits my account every two weeks?" The agent calls the take-home pay calculator, and the user sees net pay per check with federal withholding, Social Security, Medicare, Colorado tax and pre-tax deductions each on its own line. They move the 401(k) slider and watch take-home and savings change.

System diagram

- **Frontend:** Next.js web app with the profile quiz, plan dashboard, what-if sliders and deadline checklist.
- **Backend:** FastAPI server; upload, parsing and retrieval over the user's own documents, stored in SQLite; deterministic, tested calculators; the eval harness in GitHub Actions.
- **Agent:** Haiku 5.5 in a tool-calling loop. Its tools are document search and the paycheck, benefits-cost, contribution-order, ESPP, loan-vs-invest and offer-compare calculators.
- **Data it reads:** the uploaded offer letter, benefits guide and pay stub, the quiz profile, and one config file of tax-year limits with their sources.



- **Where the user comes in:** uploading documents, confirming every value the agent extracts before any calculation runs, and asking follow-up questions.
- **Hosting:** runs on localhost only, with no public web host, so uploaded documents stay on our own machines.

Why part of it is an agent

The math is a fixed workflow: taxes, plan costs, and contribution order are computed by deterministic, tested functions, never by the language model. The agent sits on top for the part a workflow cannot handle. Every employer's benefits guide uses a different layout and wording, and new hires ask open-ended questions that need several steps: find the match formula in the guide, pull salary from the offer, then call the right calculator. The agent decides which documents to search and which tools to call, then explains the result. Our evaluation will check this design against a simpler baseline: the same model answering from the documents alone, with no tools.

4. Who owns what

Each member owns one part from now to the presentation and presents it on Dec 3.

Part	Owner	Responsible for
Documents and retrieval	Ethan L.	PDF upload and parsing, pulling plan details out of benefits guides, cited answers, glossary; the SQLite store for document text, extracted fields and search index; the federal and Colorado tax config with sources; synthetic test documents.
Financial engine and agent tools	Shawn P.	Paycheck, benefits cost, contribution order, ESPP, loan-vs-invest, and offer-compare calculators; the agent loop that calls them; deeper insurance, retirement, and stock analysis scripts
User-facing app	Kaelem D.	Profile quiz, plan dashboard, what-if sliders, deadline checklist, paycheck split view; comparison charts and the chat panel, built on prebuilt components

Part	Owner	Responsible for
Evaluation, cost, and safety	Callen	Test set from the user's real questions, GitHub Actions eval, cost per task, privacy and redaction of uploaded documents; accuracy checks on each analysis against the user's judged answers; guardrails that keep real documents out of the repo and dev tools

Callen leads the conversations with our user: she has a Tyler offer herself, and Nathan is connecting us with others in the new-hire cohort. She records the right answer for each case as the user judges it and turns those into the evaluation set.

Documents and retrieval — Ethan L.

Done when: a teammate can check each one of these:

- **Extraction:** One command (python docs/ingest.py samples/) runs on a synthetic offer letter, benefits guide, and pay stub. It pulls out salary, pay frequency, premiums, deductibles, the employer HSA contribution, the 401(k) match formula, and the Employee Stock Purchase Plan (ESPP) terms. Each value matches that document's answer file and has a page number. For synthetic documents, the answer file is the spec they were generated from.
- **Retrieval:** For each of the five example questions in Appendix A, the document search tool returns the correct page among its top 3 results, labeled with document name and page.
- **Reference data:** Every value in the tax config has its source link and tax year. The config covers federal rules, Colorado withholding, Family and Medical Leave Insurance (FAMLI) and the five cities with local wage taxes.
- **Redaction:** After a synthetic pay stub is uploaded, no name, address, employee ID, or Social Security number appears in stored text or in anything sent to the model.
- **Storage:** After running extraction, one SQLite file in the git-ignored data/ folder holds each document's redacted text by page, its extracted fields with a confirmed or unconfirmed status, and the search index. Deleting the file and rerunning the command rebuilds it.

Needs from others:

- **Financial engine and agent tools:** The list of fields each calculator needs since it decides what extraction pulls out. Until then, I'll work from the inputs listed in Appendix A.
- **Evaluation, cost, and safety:** The redaction rules and test files so redaction runs inside the upload step. I'll use a placeholder redactor until then.
- **User-facing app:** A confirmation screen that shows each extracted value with a link to its page. Until then, I'll test extraction through its JSON output.

Financial engine and agent tools — Shawn P.

Done when: a teammate can check each of these:

- **Calculators**: pytest engine/ passes. The paycheck, benefits-cost, contribution-order and ESPP calculators are plain Python functions with no model calls inside. Each has tests that include its Appendix A example with a hand-checked answer. The paycheck calculator matches the IRS Publication 15-T percentage-method worked examples and Colorado withholding to within $1 per check.
- **Config only**: every tax rate and limit is read from Ethan's config file, with none hard-coded. To check: change one 401(k) limit in the config, rerun the tests, and the affected output changes.
- **Inputs list**: engine/fields.md lists every input each calculator needs, with its unit and where it comes from (which document or which quiz question). This goes to Ethan by Nov 10.
- **Agent**: python agent/run.py "<question>" --docs samples/ answers all five Appendix A questions. For each one, the log shows the right tool calls, and every dollar figure in the answer appears in a tool result. If a needed field is still unconfirmed, the agent asks for it instead of calculating.
- **Logging and baseline**: each run writes a JSON log of tool calls, arguments, results, and input and output tokens. A --no-tools flag runs the same model on the documents alone, for Callen's comparison.

- **Hard case**: for Example 4, if the ESPP has a holding period or the profile has no emergency fund, the answer does not say “always buy.”
- **Stretch**: loan-vs-invest and offer-compare follow the same rules, but only after everything above passes. This matches the cut order in Section 5.

Needs from others:

- **Documents and retrieval**: the extracted fields as JSON, each with its value, page and confirmed status, plus the tax config format settled by Nov 10. Until then, I’ll use hand-written JSON.
- **User-facing app**: the final list of quiz fields, such as filing status, savings goal, emergency fund and loans. Until then, I’ll test with three hard-coded profiles.
- **Evaluation, cost, and safety**: expected answers and an edge-case list for the five examples. Until then, I’ll compute expected answers by hand.

User-facing app — Kaelem D.

Done when: a teammate can check each of these:

- Start: npm run dev in app/ opens the Next.js app at localhost:3000, and uvicorn api.main:app starts the API. With the API off, every screen loads from stub JSON.
- API: The FastAPI server in api/ only calls functions owned by Documents and Financial engine; it holds no math. Its /docs page lists every route.
- Upload and confirm: After the synthetic documents are uploaded, a confirmation screen lists each extracted value with a page link to its redacted text. No calculator runs until its values are confirmed.
- Quiz: Six questions (filing status, city, who's covered, expected doctor visits, emergency fund, student loans) save to the profile and update the dashboard.
- Dashboard: Itemized take-home pay, recommended health plan with low- and high-use yearly cost, dollars per check to each account in order, and ESPP gain. Every figure matches the calculator output exactly.
- Sliders and checklist: The 401(k) slider updates pay and savings in under a second. The split view adds up to net pay. The checklist shows the enrollment deadline with its page link.
- Chat: Each Appendix A question gets an answer with page links and which calculator produced each number.
- Safety: Every plan screen shows the advisor disclaimer, and browser storage holds no document text.

Needs from others:

- Financial engine and agent tools: Calculators as Python functions with fixed input/output fields (field list by Nov 12), and an agent loop that streams text, tool calls and citations.
- Documents and retrieval: Per-file ingest, functions to confirm fields and fetch a page's redacted text, the enrollment deadline dates, and the glossary.
- Evaluation, cost, and safety: The disclaimer wording, and two new hires trying the quiz during interviews.

Evaluation, cost, and safety — Callen N.

Done when: a teammate can check each of these:

- **Evaluation:** one command (python eval/run.py) runs every case, the five examples plus at least 15 more from the new hires, through the agent and through the no-tools baseline, and prints a pass rate for each analysis. A case passes when every dollar figure is within $1 of the answer the user confirmed and the answer covers each point listed for that case. GitHub Actions runs it on every push to main.
- **Cost:** the same run logs input and output tokens and dollars for every question, and the report shows the average and the most expensive question for each analysis.
- **Safety:** committing a file with a fake name, address or salary is blocked; uploading a synthetic pay stub leaves no name, address, employee ID or Social Security number in anything sent to the model or saved; and a benefits guide with hidden instructions such as "ignore your rules" does not change which tools the agent calls.

Needs from others: the analysis scripts callable from a test with fixed inputs, and each question's tool calls and token counts logged (financial engine and agent tools); page numbers on every citation, and redaction run on upload before parsing (documents and retrieval); the advice disclaimer on every plan screen, and no document text kept in browser storage (user-facing app).

5. Plan to Dec 3

First end-to-end version: Tue Nov 24, a rough dry run on all five examples, before Thanksgiving.

Our user first tries it: Wed Nov 25, the day after the dry run; testing runs through Dec 2.

Week by week

Week	Who does what
Nov 9–15	Ethan: Upload and parsing done by Nov 13. Shawn: Calculators with tests, through Nov 20. Kaelem: screens from Nov 12, quiz and dashboard on stub data. Callen: the five examples turned into test cases with expected answers.
Nov 16–22	Ethan: Cited answers and plan-detail extraction. Shawn: Agent loop calling the calculators. Kaelem: Sliders and checklist wired to the engine, done Nov 23. Callen: eval set running in GitHub Actions by Nov 23; redaction on upload.
Nov 23–29	All: Tue Nov 24 dry run in lecture 25, our first end-to-end version. Wed Nov 25: users start testing. Thanksgiving Nov 26.
Nov 30–Dec 3	Users keep testing through Dec 2; each owner fixes what they find in their part; Callen reruns the eval on the new cases. Thu Dec 3: presentation.

Our biggest risks, and what we cut first

- **Advice vs education.** The app explains options and shows the math; it does not tell anyone what they must do. Every plan screen carries a short note to confirm with HR or a licensed advisor.
- **Private data, including in our own tools.** Real offer letters and pay stubs never go into the repo or into prompts in Claude Code or other dev tools. We build and test with synthetic or redacted documents, keep a git-ignored data folder, and run a check before each commit that blocks names, salaries and ID numbers.
- **Numbers that change yearly.** Tax brackets and contribution limits live in one config file with the IRS or state source and tax year next to each value.
- **Doctor network data is hard to get and often wrong.** Insurer directories vary in format and accuracy, and some don't allow scraping. If we can't get reliable network data by Nov 16, we show plan cost only and link to each insurer's own directory.
- **Scope cuts if we fall behind.** Offer comparison goes first, then student loans, then the weekly plan refresh: we would keep a fixed snapshot of plans instead. The core four features and the stock analysis are not cut.

Appendix A: Five examples in full

#	The new hire asks	Inputs the app needs	A correct answer includes
1	"My offer says 90k. What actually hits my account every two weeks?"	Offer letter, pay frequency, filing status, Colorado residence	Gross per check minus federal withholding, Social Security and Medicare, Colorado income tax, and pre-tax deductions; each line shown
2	"Should I pick the high-deductible plan with an HSA or the PPO? I rarely see a doctor."	Benefits guide premiums, deductibles, employer HSA contribution	Yearly cost under a low-use and a high-use year for each plan, and the HSA tax benefit
3	"The company matches 6% of my 401(k). Should I also open a Roth IRA, and which comes first?"	Match formula from the guide, salary, savings goal	Contribute enough to get the full match first, then the order for HSA, Roth IRA, and extra 401(k), in dollars per check
4	"We can buy company stock at a 15% discount. Should I?"	ESPP terms: discount, lookback, holding rules	The built-in gain from the discount, the risk of holding stock in your own employer, and a sell-or-hold rule of thumb
5	"I have student loans at 4.5%. Should I pay them off faster or invest?"	Loan balance and rate, match, emergency fund status	Compare the loan rate to the match and expected returns; say which dollars go where first